Jeremy Barton
Professor Neugebauer
06/28/26
DSC-640-T301

MLB Attendance Recommendation

Imports and Preprocessing

In [1]:
```
import pandas as pd
import sklearn
import spicy
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

Import the data

In [2]:
```
df = pd.read_csv('/home/nunya/Documents/GitHub/MLB-Attendance-Recommendations/dodgers-2022.csv')
```

In [3]:
```
# get some basic info about the data
df.head(5)
```

Out[3]:

	month	day	attend	day_of_week	opponent	temp	skies	day_night	cap	shirt	fireworks	bobblehead
0	APR	10	56000	Tuesday	Pirates	67	Clear	Day	NO	NO	NO	NO
1	APR	11	29729	Wednesday	Pirates	58	Cloudy	Night	NO	NO	NO	NO
2	APR	12	28328	Thursday	Pirates	57	Cloudy	Night	NO	NO	NO	NO
3	APR	13	31601	Friday	Padres	54	Cloudy	Night	NO	NO	YES	NO
4	APR	14	46549	Saturday	Padres	57	Cloudy	Night	NO	NO	NO	NO

Introduction

In this notebook, we will using data to improve the attendance of Los Angeles DOdgers Major League Baseball (MTB) games.

Data Cleaning

In this step, we will prepare the data for a machine learning algorithm. This is the most important part of the process.

Introduction

In this notebook, we will recommend which of the four items at the store, (cap, shirt, fireworks or bobblehead) to focus on marketing the most during which days of the week. This will help the Dodgers understand which items help attendees return throughout the season.

Which item sells the most will be determined by the day of the week items are sold, the number of attendees, and count of YES in each item category.

Data Cleaning

Checking for missing values.

In [4]:
```
df[df.isna().any(axis=1)]
```

Out[4]:

month	day	attend	day_of_week	opponent	temp	skies	day_night	cap	shirt	fireworks	bobblehead

It looks like there are no missing values in the data which is a good sign. Let's check for duplicates next.

Checking for duplicates.

In [5]:
```
df[df.duplicated()]
```

Out[5]:

month	day	attend	day_of_week	opponent	temp	skies	day_night	cap	shirt	fireworks	bobblehead

No duplicates either, very good.

Now we can drop any columns we don't need for the analysis. The columns we do need to predict which items to sell on what days, is `attend` , `day_of_week` , `cap` , `shirt` , `fireworks` , `bobblehead` .

In [6]:
```
sale_data = df.drop(columns=['month','day','opponent','temp','skies','day_night'])
```

Let's take a peak at it now.

In [7]:
```
sale_data.head(3)
```

Out[7]:

	attend	day_of_week	cap	shirt	fireworks	bobblehead
0	56000	Tuesday	NO	NO	NO	NO
1	29729	Wednesday	NO	NO	NO	NO
2	28328	Thursday	NO	NO	NO	NO

Nice.

Preparing the data to predict which items should be sold on what days

Looking at the data, a few things need to be adjusted so that the data can be read by machine learning models.

First, `day_of_week` should be encoded to be repsesented by integers instead of strings, and the products sold column for each item with a 'YES' or 'NO' will be converted to a 1 (YES) or 0 (NO).

In [8]:
```
# Encoding day_of_week to integer from string
label_encoder = LabelEncoder()
sale_data['day_of_week'] = label_encoder.fit_transform(sale_data['day_of_week'])

print(sale_data.head())
```

```
   attend  day_of_week cap shirt fireworks bobblehead
0   56000            5  NO    NO        NO         NO
1   29729            6  NO    NO        NO         NO
2   28328            4  NO    NO        NO         NO
3   31601            0  NO    NO       YES         NO
4   46549            2  NO    NO        NO         NO
```

Next the yeses and nos shall be encoded to 1 = YES and 0 = NO respectively.

In [9]:
```
# Encoding the binary sale indicators
label_encoder = LabelEncoder()

# Encode each categorical column
sale_data['cap'] = label_encoder.fit_transform(sale_data['cap'])
sale_data['shirt'] = label_encoder.fit_transform(sale_data['shirt'])
sale_data['fireworks'] = label_encoder.fit_transform(sale_data['fireworks'])
sale_data['bobblehead'] = label_encoder.fit_transform(sale_data['bobblehead'])
```

In [10]:
```
# Check on it again
print(sale_data.head())
```

```
   attend  day_of_week  cap  shirt  fireworks  bobblehead
0   56000            5    0      0          0           0
1   29729            6    0      0          0           0
2   28328            4    0      0          0           0
3   31601            0    0      0          1           0
4   46549            2    0      0          0           0
```

Very nice. 👍

Choosing models

At this stage we will need to choose the model(s) that will be used to predict the type of items that should be marketed the most on what days of the week.

This solution will use binary classificaiton algorithms. Specifically, linear regression, decision tree, random forest regression. They will be compared to find out which is the best model to use.

Below is the reasoning for choosing these models for this problem.

Logistic Regression

Logistic regression is an ideal baseline model for classification problems. It is fast, simple to implement, and easy to interpret through coefficients. However it can be limited in capturing complex interactions between features. This is why different models are tested; to compare and understand which problem what model is best at.

Decision Tree Classifier

Decision tree classifiers are another classification algorithm. An advantage this model has over logistic regression is that it can capture non-linear relationships and interactions between features. This is crutial for the solution because the reltionship between item characteristiscs (e.g., cap, shit) and their sales might not follow a simple linear pattern. Whereas logistic regression assumes a linear relationship between the features and the log-odds of the target variable. This could miss deeper insights that logistic regession might miss.

Random Forest Classifier

Random forests reduce overfitting by combining predictions from multiple decision trees, making them more robust against noise in the data. It provides a way to measure feature importance and can also handle complex non-linear relationships. A disadvantage is that it is more computationally expensive and less inerpretable compared to decision trees, but more than logistic regression. For the solution, random forests are valuable because they can better capture the nuanced factors affecting item sales. This could lead to improved perfomance and readability compared to individual decision trees or logistic regression.

In [11]:
```
#### Splitting the data

# Define the X and Y
X = sale_data.drop(columns=['attend'])
y = sale_data['attend']

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Trying out the models.

In [12]:
```
# Initalize the models
models = {
    'Linear Regression': LinearRegression(),
    'Decision Tree': DecisionTreeRegressor(random_state=42),
    'Random Forest': RandomForestRegressor(n_estimators=100, random_state=42)
}
```

In [13]:
```
# Pass the models through the data
#   in the sequence of training >> predictions >> evaluation

for name, model in models.items():
    # Train the model
    model.fit(X_train, y_train)

    # Make predictions on the test set
    y_pred = model.predict(X_test)

    # Evaluate the model
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    print(f"{name}: MSE={mse}, R2={r2}")
```

```
Linear Regression: MSE=81457848.32224119, R2=0.17152108926079324
Decision Tree: MSE=77168862.97726499, R2=0.2151428394048135
Random Forest: MSE=68685787.14904092, R2=0.30142119768015996
```

Now the models will be evaluated based on their performance metrics.

Methods used to evaluate will be Mean Squared Error (MSE), Root Mean Squared Error (RSME), and R2 score.

In [14]:
```
# Ealuating each model using RSME and R2

for name, model in models.items():
    y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    rmse = mse ** 0.5
    r2 = r2_score(y_test, y_pred)

    print(f"{name}: RSME={rmse}, r2={r2}")
```

```
Linear Regression: RSME=9025.400175185652, r2=0.17152108926079324
Decision Tree: RSME=8784.58097903736, r2=0.2151428394048135
Random Forest: RSME=8287.688890700527, r2=0.30142119768015996
```

HOW TO READ THIS:

Root Mean Squared Error (RSME) measures the differences between the values predicted by the model and the actual observed valued. Lower RMSEs indicate better predictive performance. For example in the results, an RMSE of 8287.69 means that on aerage, the predictions ae off my approximately that many units from the actual sales.

R-Squared (R2) is a statistical measure that explains how much of the dependent variable can be explained bt the independant variable or variables in the regression model. In the results, an R2 of .30 means that approximately 30% of the variablility in sales can be explained by the features included in the model.

In [15]:
```
# Choose the best model based on RMSE and R² scores
best_model_name = max(models.items(), key=lambda x: x[1].score(X_test, y_test))[0]
print(f"BEST PERFORMING MODEL: {best_model_name}")
```

```
BEST PERFORMING MODEL: Random Forest
```

In [16]:
```
# Final evaluation of the best model
y_pred_best = models[best_model_name].predict(X_test)
```

```
mse_best = mean_squared_error(y_test, y_pred_best)
rmse_best = mse_best ** 0.5
r2_best = r2_score(y_test, y_pred_best)
```

In [17]:
```
print(f"Best Model Performance: RMSE={rmse_best}, R²={r2_best}")
```

```
Best Model Performance: RMSE=8287.688890700527, R²=0.30142119768015996
```

Building the results

To find the recommended days of week to sell different items, feature extraction will be used which checks which features are most important for the best model.

In [18]:
```
# Extract feature importances
feature_importances = models['Random Forest'].feature_importances_
features = X_train.columns

# Creating object with features and importance
importance_df = pd.DataFrame({'Feature': features, 'Importance': feature_importances})
importance_df = importance_df.sort_values(by='Importance', ascending=False)

print("Feature Importances (Random Forest):")
print(importance_df)
```

```
Feature Importances (Random Forest):
       Feature  Importance
4   bobblehead    0.588609
0  day_of_week    0.275256
3    fireworks    0.072668
2        shirt    0.037631
1          cap    0.025835
```

Now that we know the model is focusing on the right features, we can determine which of them are most important for each day of the week.

In [19]:
```
# Determine which items are most important for each day of the week
def recommend_items_for_day(day_of_week):
    # Get the top 2-3 features for this day
    top_features = importance_df[importance_df['Feature'] != 'day_of_week'].head(3)

    return top_features

# Print recommendations for each day of the week
days_of_week = {0: "Monday", 1: "Tuesday", 2: "Wednesday", 3: "Thursday", 4: "Friday", 5: "Saturday", 6: "Sunday"}

for day in range(7):
    recommendation = recommend_items_for_day(day)
    print(f"Day of the Week: {days_of_week[day]}")
    for feature, importance in zip(recommendation['Feature'], recommendation['Importance']):
        if feature != 'day_of_week':
            print(f"Item: {feature}, Importance: {importance:.4f}")

print("\nRecommendations Summary:")
for day in range(7):
    top_items = recommend_items_for_day(day)
    top_features_str = ', '.join(top_items['Feature'].tolist())
    print(f"Day of the Week: {days_of_week[day]}, Top Items to Sell: {top_features_str}")
```

```
Day of the Week: Monday
Item: bobblehead, Importance: 0.5886
Item: fireworks, Importance: 0.0727
Item: shirt, Importance: 0.0376
Day of the Week: Tuesday
Item: bobblehead, Importance: 0.5886
Item: fireworks, Importance: 0.0727
Item: shirt, Importance: 0.0376
Day of the Week: Wednesday
Item: bobblehead, Importance: 0.5886
Item: fireworks, Importance: 0.0727
Item: shirt, Importance: 0.0376
Day of the Week: Thursday
Item: bobblehead, Importance: 0.5886
Item: fireworks, Importance: 0.0727
Item: shirt, Importance: 0.0376
Day of the Week: Friday
Item: bobblehead, Importance: 0.5886
Item: fireworks, Importance: 0.0727
Item: shirt, Importance: 0.0376
Day of the Week: Saturday
Item: bobblehead, Importance: 0.5886
Item: fireworks, Importance: 0.0727
Item: shirt, Importance: 0.0376
Day of the Week: Sunday
Item: bobblehead, Importance: 0.5886
Item: fireworks, Importance: 0.0727
Item: shirt, Importance: 0.0376

Recommendations Summary:
Day of the Week: Monday, Top Items to Sell: bobblehead, fireworks, shirt
Day of the Week: Tuesday, Top Items to Sell: bobblehead, fireworks, shirt
Day of the Week: Wednesday, Top Items to Sell: bobblehead, fireworks, shirt
Day of the Week: Thursday, Top Items to Sell: bobblehead, fireworks, shirt
Day of the Week: Friday, Top Items to Sell: bobblehead, fireworks, shirt
Day of the Week: Saturday, Top Items to Sell: bobblehead, fireworks, shirt
Day of the Week: Sunday, Top Items to Sell: bobblehead, fireworks, shirt
```

Acknowledging an issue with the recommendation

The recommendation step above returns the same three items (bobblehead, fireworks, shirt) for every day of the week. That is not a true per-day recommendation. Two things cause it:

1. The feature importances are global. they are calculated once across all games, so there is only one ranking to report.
2. The `recommend_items_for_day()` function takes a `day_of_week` argument but never uses it, so every day returns that same global top-3.

There is also a deeper limitation. A feature importance only tells us how much the model *relies* on a column; it does not tell us whether offering that item **raises or lowers** attendance, or by how much. To make a recommendation management can act on, I will measure each item's effect on attendance directly.

Measuring each item's effect on attendance

For each of the four items, I compare the average attendance of games that offered that giveaway against games that did not. This uses all 81 games and answers the business question directly: how many more (or fewer) people show up when each item is offered?

In [20]:
```python
# Compare average attendance with vs. without each giveaway item
items = ['cap', 'shirt', 'fireworks', 'bobblehead']
overall = df['attend'].mean()
print(f"Overall average attendance: {overall:,.0f}\n")

rows = []
for item in items:
    with_item = df[df[item] == 'YES']['attend']
    without_item = df[df[item] == 'NO']['attend']
    rows.append({
        'Item': item,
        'Games offered': len(with_item),
        'Avg WITH': round(with_item.mean()),
        'Avg WITHOUT': round(without_item.mean()),
        'Lift': round(with_item.mean() - without_item.mean())
    })

lift_df = pd.DataFrame(rows).sort_values('Lift', ascending=False)
print(lift_df.to_string(index=False))
```

```
Overall average attendance: 41,040

      Item  Games offered  Avg WITH  Avg WITHOUT  Lift
bobblehead             11     53145        39138 14007
     shirt              3     46644        40825  5819
 fireworks             14     41078        41032    46
       cap              2     38190        41112 -2923
```

A caveat: items are tied to specific days

Before recommending, it is worth checking which days each item is actually offered on. If an item only ever runs on one day, its apparent effect could really be the effect of that day rather than the item itself.

In [21]:
```
# Check which days each item is offered on
for item in items:
    days_offered = df[df[item] == 'YES']['day_of_week'].value_counts().to_dict()
    print(f"{item:11s} offered on: {days_offered}")
```

```
cap         offered on: {'Tuesday': 1, 'Sunday': 1}
shirt       offered on: {'Sunday': 1, 'Monday': 1, 'Tuesday': 1}
fireworks   offered on: {'Friday': 13, 'Wednesday': 1}
bobblehead  offered on: {'Tuesday': 6, 'Saturday': 2, 'Thursday': 2, 'Sunday': 1}
```

Results and recommendation

Reading the lift table alongside the day check:

- Bobblehead is the clear driver. Games with a bobblehead average about +14,000 attendees over games without one, across 11 games, which the largest and best-supported effect in the data. Bobbleheads run mostly on Tuesdays (6 of 11), with the rest spread over Thursday, Saturday, and Sunday, so the effect is not just one day.
- Fireworks show almost no lift (~+50). Fireworks are offered on Fridays in 13 of their 14 games, so what looked important in the model was really the Friday games themselves, not the fireworks.
- Shirt shows a positive lift (~+5,800) but on only 3 games, so the evidence is thin and promising, but not something to bank on.
- Cap shows a small negative lift on only 2 games which is too little data to draw a conclusion.

Recommendation to management: concentrate giveaway investment on **bobbleheads**, which is the only item with a large, well-supported attendance effect. Because bobbleheads have mostly run on Tuesdays, testing them on additional days, particularly weekends, would help confirm the lift is from the item and not the day. The earlier "same three items every day" output was a code artifact rather than a real pattern, and the prominence of fireworks reflected the day it runs on rather than the item itself.

Assumptions / notes: attendance lift is a simple group comparison, not a controlled estimate, and giveaway days are not randomly assigned, so item and day effects are partly entangled. With only 81 games and a handful of instances for some items, a larger sample across multiple seasons would be needed to firm up the cap and shirt findings.